KUBERNETES CI/CD

EXTREMELY DETAILED BEGINNER GUIDE

A click-by-click, copy-and-paste walkthrough for Windows, VS Code, GitHub, GitHub Actions, Docker Hub, and Docker Desktop Kubernetes.

Goal: Follow the instructions even if you are new to these tools. The guide explains where to click, what to type, what each step means, what success looks like, and what to do when something goes wrong.

Stage	What you will achieve
1. Check setup	Confirm Git, Python, Docker, kubectl, and Kubernetes are working.
2. Create app	Create a small Python Flask website and automated tests.
3. Run tests	Verify the app works on your own computer.
4. Docker	Build an image and run it as a container.
5. GitHub	Upload the project to a GitHub repository.
6. GitHub Actions	Automatically run tests after pushing code.
7. Docker Hub	Automatically publish a Docker image after tests pass.
8. Kubernetes	Run two copies of the app and open it in a browser.
9. Update	Publish and deploy a new application version.

Important: Do the steps in order. Do not skip a checkpoint just because a command looks familiar.

How to use this guide

• Keep this PDF open beside VS Code and your browser.

• Follow one numbered step at a time. Do not paste all commands at once unless the guide explicitly groups them.

• When a step says to create a file, click that file in VS Code and paste only the content shown for that file.

• When a step says to run a command, paste it into the VS Code PowerShell terminal, not into a Python or YAML file.

• Replace every placeholder such as YOUR_DOCKERHUB_USERNAME with your real value. Do not type the placeholder literally.

• If your screen looks different, stop and compare the heading and filename before continuing.

What we are building

A small website with two URLs: the home page shows “Hello Students! Kubernetes CI/CD is working!” and /health returns a healthy status. GitHub Actions will test it. If tests pass on a push to main, the workflow will build a Docker image and publish it to Docker Hub. You will then deploy that image to Kubernetes on your computer.

Words you will see

Word	Simple meaning
Repository (repo)	The online project folder on GitHub.
Commit	A saved checkpoint of code changes.
Push	Upload commits from your computer to GitHub.
Workflow	Instructions GitHub Actions follows.
Job / step	A job is a group of work; a step is one action inside it.
Image	A packaged application template.
Container	A running copy created from an image.
Pod	The Kubernetes unit that runs the container.
Deployment	Tells Kubernetes how many copies to keep running.
Service	Provides a stable way to reach the app.

Part 1 — Check your tools

Step 1. Open the correct terminal

1. Open Visual Studio Code.

2. At the top menu, click **Terminal**.

3. Click **New Terminal**.

4. Look at the terminal panel. The prompt should look like PowerShell and may start with `PS C:\...`.

Step 2. Check each tool

Copy and run each command separately. After each command, check whether a version or details are shown.

```
git --version
python --version
docker --version
docker info
kubectl version --client
kubectl get nodes
```

Expected: Git and Python show version numbers; Docker prints a version and engine details; kubectl prints its client version; kubectl get nodes shows a node with STATUS Ready.

If Python is not found: install Python 3 from the official Python website, select the option to add Python to PATH if shown, then close and reopen VS Code.

If Docker info reports a connection error: open Docker Desktop from the Windows Start menu and wait until it has finished starting.

If kubectl get nodes fails or the node is not Ready: open Docker Desktop → Settings → Kubernetes (the exact menu may vary by version) → enable/start Kubernetes, apply changes if prompted, and wait. Run kubectl get nodes again.

Do not continue until Docker works and Kubernetes shows Ready.

Part 2 — Create the project folder

Step 3. Create and open the folder

In VS Code, click File → Open Folder. In the folder picker, choose a location you can find easily, such as Documents. Create a new folder named student-k8s-demo, select it, and click Select Folder. If Windows asks whether you trust the folder, choose the option to trust it because this is your own project.

Alternative: if you are already in a PowerShell terminal at the location where you want the project, run the following commands. Do not run mkdir if you already created the folder.

```
mkdir student-k8s-demo
cd student-k8s-demo
code .
```

If the last command is not recognized, use VS Code → File → Open Folder instead.

Step 4. Create files in VS Code Explorer

1. On the far-left activity bar, click the Explorer icon (it looks like two sheets of paper).

2. At the top of the Explorer panel, find the project folder name.

3. Move your pointer over the folder name. Click the New File icon to create app.py.

4. Repeat to create requirements.txt, test_app.py, Dockerfile, .dockerignore, and .gitignore.

5. Click the New Folder icon and create a folder named k8s. Inside it create deployment.yaml and service.yaml.

6. Create a folder named .github. Inside it create a folder named workflows. Inside workflows create pipeline.yml.

7. If you cannot see a file whose name begins with a dot, use the Explorer; do not add an extra extension such as .txt.

Your project structure should look like this:

```
student-k8s-demo/
  app.py
  requirements.txt
  test_app.py
  Dockerfile
  .dockerignore
  .gitignore
  k8s/
    deployment.yaml
    service.yaml
  .github/
    workflows/
      pipeline.yml
```

Part 3 — Add the application and tests

Step 5. Put code in app.py

In Explorer, click app.py. If it is empty, paste the following code. Press Ctrl+S to save.

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello Students! Kubernetes CI/CD is working!"

@app.route("/health")
def health():
    return jsonify(status="healthy"), 200

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 6. Put dependencies in requirements.txt

Click requirements.txt, paste the following two lines, and press Ctrl+S.

```
Flask>=3.0,<4.0
pytest>=8.0,<9.0
```

Flask is the web framework. pytest is the test runner.

Step 7. Put tests in test_app.py

```
from app import app

def test_home_page():
    client = app.test_client()
    response = client.get("/")
    assert response.status_code == 200
    assert b"Kubernetes CI/CD is working" in response.data

def test_health_endpoint():
    client = app.test_client()
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json["status"] == "healthy"
```

Save the file with Ctrl+S. These tests check that the home page responds successfully and the health endpoint returns healthy.

Part 4 — Run the app and tests locally

Step 8. Install Python packages

Click Terminal → New Terminal. Confirm the terminal is inside the student-k8s-demo folder. Run:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

The first command creates an isolated Python environment named .venv. The second installs the packages without needing to activate the environment.

Step 9. Run the tests

```
.\.venv\Scripts\python.exe -m pytest -v
```

Expected: the terminal reports 2 passed. If it says pytest is not found, repeat the install command from Step 8 and inspect any error.

Step 10. Start the app

```
.\.venv\Scripts\python.exe app.py
```

Keep this terminal open. Open a browser tab and enter http://127.0.0.1:5000/. You should see the welcome message. Then open http://127.0.0.1:5000/health. You should see JSON containing healthy.

To stop the app, return to the terminal running app.py and press Ctrl+C. Do not close the entire project.

Part 5 — Configure Docker

Step 11. Create Dockerfile

Click the file named Dockerfile (capital D, no .txt extension), paste this content, and save.

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

Step 12. Create .dockerignore

```
.venv/
__pycache__/
.pytest_cache/
.git/
```

Step 13. Create .gitignore

```
.venv/
__pycache__/
.pytest_cache/
.env
```

The ignore files prevent local environment folders and private environment files from being included where they do not belong. Never put passwords or tokens in the project.

Step 14. Build the Docker image

In the terminal, ensure you are in the project folder, then run:

```
docker build -t student-k8s-demo:v1 .
docker images
```

The dot at the end of docker build is important. It means “use the current folder as the build context.” The image list should include student-k8s-demo with tag v1.

Step 15. Run and test the container

```
docker run --rm -d --name student-demo -p 5000:5000 student-k8s-demo:v1
docker ps
docker logs student-demo
```

Open http://localhost:5000/ in the browser. It should show the welcome message. Then stop the container:

```
docker stop student-demo
```

If port 5000 is already in use, use this alternative command and browse to http://localhost:5001/:

```
docker run --rm -d --name student-demo -p 5001:5000 student-k8s-demo:v1
```

Part 6 — Create the GitHub repository

Step 16. Create the repository in your browser

1. Open https://github.com/ and sign in.

2. At the top-right, click the + button, then click New repository. If the menu differs, open the repositories page and select New.

3. In Repository name, enter student-k8s-demo. Use this name consistently in the guide.

4. Choose Public for this learning exercise so the local Kubernetes cluster can pull the image without extra private-image setup. Do not put sensitive information in a public repository.

5. Leave Add a README file unchecked. Leave .gitignore and license set to None if those options are shown, because you already created local files.

6. Click Create repository.

7. On the next page, find the HTTPS repository URL. It will look like https://github.com/YOUR_GITHUB_USERNAME/student-k8s-demo.git. Keep it available.

If you already created a repository with a different name, use that repository's exact URL and do not create a duplicate unless you want a separate project.

Step 17. Upload your local code

Return to VS Code. Open a terminal in the project folder and run these commands one at a time. Replace the URL with the exact HTTPS URL of your repository.

```
git init
git add .
git commit -m "Initial Flask application and Docker setup"
git branch -M main
git remote add origin https://github.com/YOUR_GITHUB_USERNAME/student-k8s-demo.git
git push -u origin main
```

Git may open a browser sign-in or authentication prompt. Follow the official sign-in flow. Do not put your GitHub password or token into a source file.

If Git says remote origin already exists, do not run git remote add origin again. Run git remote -v to see the existing URL. If it is wrong, use git remote set-url origin with your repository URL.

Step 18. Verify the upload

In your browser, open the repository and click Code. The files and folders should be visible. Click .github, then workflows, then pipeline.yml to verify that the workflow file was uploaded.

Part 7 — Create the GitHub Actions workflow

Step 19. Understand the Actions page

On your repository page, the top navigation includes Code, Issues, Pull requests, Actions, Projects, and other tabs. Click Actions. If you see “Get started with GitHub Actions” and suggested workflow cards, GitHub has not yet detected a workflow on the default branch.

Step 20. Create a workflow using the browser editor

1. On the Actions page, click the blue link “set up a workflow yourself” near the top. This is the route shown in the screenshot you shared.

2. GitHub opens a file editor. The filename field should show a path similar to .github/workflows/main.yml. That is a valid workflow path. You can keep main.yml, or use pipeline.yml; the filename itself does not control the workflow's displayed name.

3. Click inside the editor, press Ctrl+A to select the default code, and paste the YAML below.

4. Check indentation carefully. YAML uses spaces; do not use tabs. The name line starts at the left edge, and nested lines are indented.

5. Click Commit changes… near the upper-right of the editor.

6. In the confirmation dialog, keep the option to commit directly to the main branch if offered, enter a short commit message if requested, and click Commit changes.

7. After committing, click the Actions tab again. A run should appear if the workflow is valid and the event triggers it.

Step 21. Paste this CI-only workflow first

This first workflow only runs tests. That is intentional: get this working before adding Docker publishing.

```
name: Student Kubernetes CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read

jobs:
  test:
    name: Checkout and Test
    runs-on: ubuntu-latest

    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run automated tests
        run: python -m pytest -v
```

Step 22. Read the Actions result

• Green check mark: the workflow completed successfully.

• Yellow circle: the workflow is running or waiting.

• Red cross: a step failed. Click the run, then click the failed job (Checkout and Test), then expand the red step to read the error.

• No workflow shown: check that the file is on the default branch, the file is inside .github/workflows/, and the commit was saved.

If the file was created in GitHub's browser editor, you do not need to run git push for that same change; GitHub has already committed it online. If you edit locally instead, save, commit, and push.

Part 8 — Create a Docker Hub repository and token

Step 23. Create the Docker Hub repository

1. Open https://hub.docker.com/ and sign in, or create an account if needed.

2. Click Repositories in the navigation, then click Create repository (the exact layout can vary).

3. Enter student-k8s-demo as the repository name.

4. Choose Public for this introductory exercise. A public image can be pulled by the local cluster without configuring a Kubernetes image-pull secret. Do not publish private or sensitive application data.

5. Click Create.

Step 24. Create a Docker Hub access token

1. Click your profile/avatar in Docker Hub.

2. Open Account settings or My Account, then look for Personal access tokens or Access tokens. Docker Hub's interface can change; use the account settings section if the label differs.

3. Click Generate new token.

4. For the description, enter GitHub Actions.

5. Choose Read & Write permissions if offered. This allows the workflow to upload images to your repository. Use the minimum permissions that allow pushing images.

6. Click Generate.

7. Copy the token immediately and store it in a secure place. The token may not be shown again. Do not send it to anyone or paste it into this chat.

The token is a credential. It gives GitHub Actions permission to sign in to Docker Hub. It is not the same as the Docker Hub username.

Step 25. Add GitHub repository secrets

1. Open your GitHub repository in the browser.

2. Click Settings in the repository navigation (not your personal account settings).

3. In the left sidebar, find Security, then click Secrets and variables to expand it.

4. Click Actions.

5. Click New repository secret.

6. In Name, type exactly DOCKERHUB_USERNAME. In Secret, type your Docker Hub username. Click Add secret.

7. Click New repository secret again.

8. In Name, type exactly DOCKERHUB_TOKEN. In Secret, paste the Docker Hub access token. Click Add secret.

The secret names are case-sensitive for practical purposes: use the exact uppercase names shown. You should see the secret names listed, but GitHub will not show their saved values.

Part 9 — Add Docker image publishing to Actions

Step 26. Replace the workflow content

Open .github/workflows/pipeline.yml in VS Code or the workflow file in GitHub. Replace the complete contents with the code below. Do not paste this underneath the old workflow; replace the old content so there is one clean workflow.

```
name: Student Kubernetes CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read

jobs:
  test:
    name: Checkout and Test
    runs-on: ubuntu-latest
    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run automated tests
        run: python -m pytest -v

  docker:
    name: Build and Push Docker Image
    needs: test
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build and push Docker image
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${{ secrets.DOCKERHUB_USERNAME }}/student-k8s-demo:${{ github.sha }}
```

Step 27. Save and trigger the workflow

If editing in VS Code, press Ctrl+S, then run:

```
git add .github/workflows/pipeline.yml
git commit -m "Add Docker build and publish stage"
git push origin main
```

If Git reports nothing to commit, check whether the workflow content is already committed. If you edited the file directly on GitHub, use its Commit changes button instead.

Step 28. Confirm the image was published

1. In GitHub, open Actions and click the newest run.

2. The test job should pass before the Docker job begins. The Docker job depends on test.

3. Open the Docker job and check the Login to Docker Hub and Build and push Docker image steps.

4. If both jobs succeed, open Docker Hub → Repositories → student-k8s-demo.

5. Find the image tag. It is the long commit SHA shown in the successful workflow run. Copy the full SHA or use the exact tag shown on Docker Hub.

If Docker login fails, re-check the secret names and the token permissions. If image publishing fails, open the failed step and read the exact error.

Part 10 — Configure Kubernetes

Step 29. Find the image tag

The workflow publishes an image whose tag is the Git commit SHA. In GitHub Actions, open the successful run for the push you want to deploy. The commit SHA is a long hexadecimal string, usually 40 characters. Docker Hub's Tags tab can also show it.

Your image name will look like this: YOUR_DOCKERHUB_USERNAME/student-k8s-demo:COMMIT_SHA. Replace both placeholders with your real username and the exact published SHA.

Step 30. Fill in k8s/deployment.yaml

Open k8s/deployment.yaml in VS Code. Replace the placeholder image value with your actual Docker Hub username and commit SHA. Keep the rest of the YAML unchanged.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: student-k8s-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: student-k8s-demo
  template:
    metadata:
      labels:
        app: student-k8s-demo
    spec:
      containers:
        - name: student-k8s-demo
          image: YOUR_DOCKERHUB_USERNAME/student-k8s-demo:REPLACE_WITH_COMMIT_SHA
          ports:
            - containerPort: 5000
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 5
```

Example format only: if your Docker Hub username were madhanuser and the tag were abc123, the image field would be madhanuser/student-k8s-demo:abc123. Do not copy that example unless it matches your real account and tag.

Step 31. Fill in k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: student-k8s-demo
spec:
  type: NodePort
  selector:
    app: student-k8s-demo
  ports:
    - port: 80
      targetPort: 5000
      nodePort: 30080
```

The container listens on port 5000. The Service forwards traffic to it and exposes NodePort 30080 for local browser access.

Step 32. Apply Kubernetes resources

Save both YAML files. In VS Code terminal, make sure you are inside the project folder and your Docker Desktop Kubernetes context is active.

```
kubectl config current-context
kubectl get nodes
kubectl apply -f k8s/deployment.yaml
```

```
kubectl apply -f k8s/service.yaml
kubectl rollout status deployment/student-k8s-demo
kubectl get deployments
kubectl get pods
kubectl get services
```

The context should be Docker Desktop's Kubernetes context (often docker-desktop). The deployment should show 2 desired/ready replicas, and the Pods should eventually show Running and Ready.

Step 33. Open the application

Open these URLs in a browser:

```
http://localhost:30080/
http://localhost:30080/health
```

The first URL should show the welcome message. The second should show JSON with status healthy.

Part 11 — Update the application

Step 34. Change the home-page message

Open app.py and replace the return line inside home() with:

```
return "Version 2: Updated through CI/CD!"
```

Save the file. Run tests locally if you want to verify before pushing:

```
.\.venv\Scripts\python.exe -m pytest -v
```

Step 35. Commit and push the update

```
git add app.py
git commit -m "Update application message"
git push origin main
```

Wait for GitHub Actions to pass. A new Docker image will be published using the new commit SHA.

Step 36. Deploy the new image

Copy the new commit SHA from the successful Actions run. Replace YOUR_DOCKERHUB_USERNAME and NEW_COMMIT_SHA in the command below, then run it in the VS Code terminal.

```
kubectl set image deployment/student-k8s-demo student-k8s-demo=YOUR_DOCKERHUB_USERNAME/student-k8s-demo:NEW_COMMIT_SHA
kubectl rollout status deployment/student-k8s-demo
kubectl get pods
```

Refresh http://localhost:30080/. You should now see the Version 2 message.

Important: in this beginner setup, tests and image publishing are automated, but the local Kubernetes deployment is manual. GitHub-hosted runners normally cannot reach Kubernetes running only on your own laptop.

Part 12 — Troubleshooting

Problem	What to do
docker command not found	Start/install Docker Desktop; reopen VS Code terminal.
docker info connection error	Start Docker Desktop and wait until the engine is running.
kubectl get nodes fails	Check Docker Desktop Kubernetes and current context.
Git push rejected	Check repository URL, sign-in, branch, and whether the remote was initialized with a README.
Actions workflow missing	Confirm YAML file is committed to the default branch and is inside .github/workflows/.
Actions tests fail	Open the run → failed job → failed step. Fix the exact error before adding Docker.
Docker login fails	Check DOCKERHUB_USERNAME and DOCKERHUB_TOKEN repository secrets and token permissions.
ImagePushBackOff / ImagePullBackOff	Check the image name/tag, that the image exists, and that the repository is public.
Pod Pending	Run kubectl describe pod POD_NAME and read the Events section.
CrashLoopBackOff	Run kubectl logs deployment/student-k8s-demo and inspect startup errors.
Browser cannot connect	Check Pod readiness, Service, port 30080, and kubectl get services.
Old version still appears	Confirm the new SHA was deployed, wait for rollout, then hard-refresh the browser.

Useful commands

```
kubectl get all
kubectl describe deployment student-k8s-demo
kubectl get pods
kubectl logs deployment/student-k8s-demo
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl describe service student-k8s-demo
```

Final checklist

Tick each item only after you have verified it.

■ Git, Python, Docker, and kubectl commands work.

■ kubectl get nodes shows Ready.

■ The Flask app opens locally.

■ Both local tests pass.

■ Docker image builds and container runs.

■ Source code is visible in GitHub.

■ GitHub Actions workflow appears and tests pass.

■ Docker Hub repository exists.

■ DOCKERHUB_USERNAME and DOCKERHUB_TOKEN secrets exist in GitHub.

■ The Docker build-and-push job succeeds.

■ The image tag is visible in Docker Hub.

■ Kubernetes Deployment has two ready Pods.

■ The Service exists and the browser can reach localhost:30080.

■ A new code version is built and manually deployed successfully.

Safety reminders

• Never put access tokens, passwords, or other secrets into source files.

• Never paste your Docker token into a public GitHub issue, a screenshot, or this chat.

• Use a public image only for this non-sensitive learning demo. Private images need Kubernetes image-pull credentials.

• If you see a different UI label, look for the same purpose in Settings or the left navigation; online interfaces can change.

• If a command fails, keep the exact error message. It is usually the fastest way to diagnose the problem.

End of guide. Work through one checkpoint at a time; you do not need to understand every tool before beginning.